

C Programming

[INDEX](#)[◀ Previous](#)[Next ▶](#)

PREPROCESSOR STATEMENTS

The **define** statement is used to make programs more readable. Consider the following examples,

```
#define TRUE      1      /* Don't use a semi-colon , # must be first characte
#define FALSE     0
#define NULL      0
#define AND       &
#define OR        |
#define EQUALS    ==

game_over = TRUE;
while( list_pointer != NULL )
    .....
```

Note that preprocessor statements begin with a # symbol, and are NOT terminated by a semi-colon. Traditionally, preprocessor statements are listed at the beginning of the source file.

Preprocessor statements are handled by the compiler (or preprocessor) before the program is actually compiled. All # statements are processed first, and the symbols (like TRUE) which occur in the C program are replaced by their value (like 1). Once this substitution has taken place by the preprocessor, the program is then compiled.

In general, preprocessor constants are written in **UPPERCASE**.

[Click](#) here for more information of preprocessor statements, including [macros](#).

Class Exercise C4

Use pre-processor statements to replace the following constants

```
0.312
W
37
```

[Click here for answers](#)

LITERAL SUBSTITUTION OF SYMBOLIC CONSTANTS USING #define

Lets now examine a few examples of using these symbolic constants in our programs. Consider the following program which defines a constant called TAX_RATE.

```
#include <stdio.h>

#define TAX_RATE 0.10

main()
```

```
{
    float balance;
    float tax;

    balance = 72.10;
    tax = balance * TAX_RATE;
    printf("The tax on %.2f is %.2f\n", balance, tax );
}
```

The pre-processor first replaces all symbolic constants before the program is compiled, so after preprocessing the file (and before its compiled), it now looks like,

```
#include <stdio.h>

#define TAX_RATE 0.10

main()
{
    float balance;
    float tax;

    balance = 72.10;
    tax = balance * 0.10;
    printf("The tax on %.2f is %.2f\n", balance, tax );
}
```

YOU CANNOT ASSIGN VALUES TO THE SYMBOLIC CONSTANTS

Considering the above program as an example, look at the changes we have made below. We have added a statement which tries to change the TAX_RATE to a new value.

```
#include <stdio.h>

#define TAX_RATE 0.10

main()
{
    float balance;
    float tax;

    balance = 72.10;
    TAX_RATE = 0.15;
    tax = balance * TAX_RATE;
    printf("The tax on %.2f is %.2f\n", balance, tax );
}
```

This is **illegal**. You cannot re-assign a new value to a symbolic constant.

ITS LITERAL SUBSTITUTION, SO BEWARE OF ERRORS

As shown above, the preprocessor performs literal substitution of symbolic constants. Lets modify the previous program slightly, and introduce an error to highlight a problem.

```
#include <stdio.h>

#define TAX_RATE 0.10;

main()
{
    float balance;
    float tax;

    balance = 72.10;
    tax = (balance * TAX_RATE )+ 10.02;
    printf("The tax on %.2f is %.2f\n", balance, tax );
}
```

In this case, the error that has been introduced is that the *#define* is terminated with a semi-colon. The preprocessor performs the substitution and the offending line (which is flagged as an error by the compiler) looks like

```
tax = (balance * 0.10; )+ 10.02;
```

However, you do not see the output of the preprocessor. If you are using TURBO C, you will only see

```
tax = (balance * TAX_RATE )+ 10.02;
```

flagged as an error, and this actually looks okay (but its not! after substitution takes place).

MAKING PROGRAMS EASY TO MAINTAIN BY USING *#define*

The whole point of using *#define* in your programs is to make them easier to read and modify. Considering the above programs as examples, what changes would you need to make if the TAX_RATE was changed to 20%.

Obviously, the answer is once, where the *#define* statement which declares the symbolic constant and its value occurs. You would change it to read

```
#define TAX_RATE = 0.20
```

Without the use of symbolic constants, you would hard code the value 0.20 in your program, and this might occur several times (or tens of times).

This would make changes difficult, because you would need to search and replace every occurrence in the program. However, as the programs get larger, **what would happen if you actually used the value 0.20 in a calculation that had nothing to do with the TAX_RATE!**

SUMMARY OF *#define*

- allow the use of symbolic constants in programs
- in general, symbols are written in uppercase
- are not terminated with a semi-colon
- generally occur at the beginning of the file

- each occurrence of the symbol is replaced by its value
 - makes programs readable and easy to maintain
-

©Copyright B Brown. 1984-1999. All rights reserved.

